

Highly Available WordPress Deployment on AWS Using CloudFormation

Anish Kumar Vaswani

Abstract

This document forms part of the assessment for SIT233 Cloud Computing. This report presents the design, deployment, and evaluation of a highly available WordPress application hosted on Amazon Web Services (AWS). The solution integrates multiple AWS services including Amazon EC2, Amazon RDS, Amazon S3, Application Load Balancer, Auto Scaling Groups, Virtual Private Cloud (VPC), and AWS CloudFormation to improve scalability, fault tolerance, availability, and infrastructure maintainability. The infrastructure was provisioned using Infrastructure as Code (IaC) principles by separating the networking and application resources into independent CloudFormation stacks. The final architecture demonstrates load-balanced WordPress access through an Application Load Balancer, automatic EC2 instance replacement using Auto Scaling, secure private subnet deployment for the database layer, Multi-AZ and read replica configurations for Amazon RDS, and media file offloading to Amazon S3. The completed system provides a resilient and scalable cloud-based web hosting environment compared to a traditional single-server deployment.

Index Terms

Cloud Computing, Amazon Web Services, WordPress, CloudFormation, Infrastructure as Code, Auto Scaling, High Availability

I. INTRODUCTION

Cloud computing has transformed the way modern applications are deployed, managed, and scaled by providing on-demand access to computing resources over the internet. Instead of relying on traditional on-premises infrastructure, organisations can utilise cloud platforms to achieve higher availability, improved scalability, enhanced fault tolerance, and reduced operational overhead. Amazon Web Services (AWS) is one of the leading cloud service providers and offers a broad range of managed services that support modern web application deployment and infrastructure automation [1].

This project focuses on the deployment of a highly available WordPress application using AWS cloud services. WordPress is one of the most widely used content management systems (CMS) for website and blog development due to its flexibility, extensive plugin ecosystem, and ease of management [2]. However, deploying WordPress on a single virtual machine introduces several limitations including single points of failure, limited scalability, and reduced reliability. To overcome these limitations, this project implements a cloud-based architecture that distributes resources across multiple Availability Zones and integrates automated infrastructure management.

The implemented solution utilises several AWS services including Amazon EC2, Amazon RDS, Amazon S3, Application Load Balancer (ALB), Auto Scaling Groups (ASG), Virtual Private Cloud (VPC), and AWS CloudFormation. The networking layer was designed using public and private subnets with Internet Gateway (IGW), NAT Gateway, and route table configurations to provide secure communication between infrastructure components. The application layer includes load-balanced WordPress web servers, a highly available relational database deployment, automatic instance scaling, and media storage integration through Amazon S3.

Infrastructure as Code (IaC) principles were implemented using AWS CloudFormation to automate infrastructure provisioning and improve deployment reproducibility [3]. The infrastructure was separated into networking and application stacks to support modular deployment and simplified lifecycle management. In addition, high availability testing was performed by terminating EC2 instances and validating automatic replacement through Auto Scaling Groups.

The primary objective of this project is to design and implement a secure, scalable, and fault-tolerant WordPress hosting environment on AWS while demonstrating practical understanding of cloud infrastructure deployment, automation, scalability, and high availability principles.

II. SYSTEM ARCHITECTURE AND INFRASTRUCTURE DESIGN

The system architecture for this project was designed to provide a highly available, scalable, secure, and fault-tolerant WordPress hosting environment using Amazon Web Services (AWS). The overall infrastructure separates networking and application resources into multiple layers to improve reliability, maintainability, and resource isolation. The architecture follows modern cloud deployment principles by distributing services across multiple Availability Zones while integrating managed AWS services to reduce administrative overhead.

The networking layer was implemented using a custom Virtual Private Cloud (VPC) consisting of both public and private subnets. Public subnets were used to host internet-facing resources including the Application Load Balancer (ALB) and NAT

Gateway, while private subnets were used to host the WordPress web servers and Amazon RDS database instances. Internet access for public resources was provided through an Internet Gateway (IGW), whereas outbound internet communication from private subnet resources was handled through the NAT Gateway. Route tables were configured to ensure secure and controlled communication between all infrastructure components.

The application layer was designed around a load-balanced WordPress deployment using Amazon EC2 instances managed by an Auto Scaling Group (ASG). Incoming HTTP traffic is distributed through the Application Load Balancer, which forwards requests to healthy EC2 instances registered within the target group. Auto Scaling was configured to automatically increase or decrease the number of EC2 instances based on CPU utilisation thresholds, improving scalability and ensuring continued availability during varying workloads.

The database layer was implemented using Amazon Relational Database Service (RDS) with a MySQL engine deployment configured in private subnets. High availability was achieved by enabling Multi-AZ deployment and creating a read replica in a separate Availability Zone. This configuration improves fault tolerance, database redundancy, and read performance while protecting the database from direct public internet exposure.

Amazon S3 was integrated into the architecture to provide scalable object storage for WordPress media uploads. Media files uploaded through the WordPress application were offloaded directly to the S3 bucket using the Offload Media plugin, reducing dependency on local EC2 storage and improving storage scalability.

Infrastructure provisioning and deployment automation were implemented using AWS CloudFormation. The infrastructure was separated into two independent CloudFormation stacks: a networking stack and an application stack. This modular approach improves deployment reproducibility, infrastructure management, and lifecycle separation between networking and application resources.

Figure 1 illustrates the overall system architecture and the interaction between the AWS services used throughout the project.

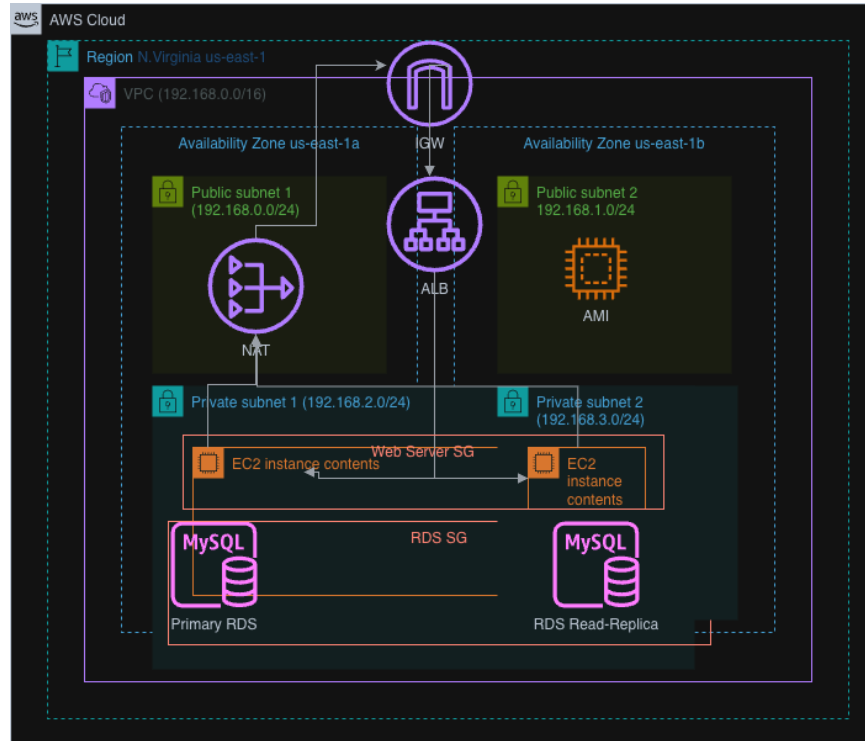


Fig. 1. Highly Available WordPress Architecture on AWS

III. IMPLEMENTATION

This section presents the implementation of the highly available WordPress infrastructure on Amazon Web Services (AWS). The deployment integrated multiple AWS services including Amazon EC2, Amazon RDS, Amazon S3, Application Load Balancer (ALB), Auto Scaling Groups (ASG), Virtual Private Cloud (VPC), and AWS CloudFormation. The infrastructure was designed to improve scalability, fault tolerance, security, and deployment automation while following cloud computing best practices.

A. VPC and Networking Configuration

The networking infrastructure was implemented using a custom Amazon VPC configured with the CIDR block 192.168.0.0/16. The VPC was divided into two public subnets and two private subnets distributed across multiple Availability Zones to improve fault tolerance and availability [4].

Public subnets hosted internet-facing services including the Application Load Balancer and NAT Gateway, while private subnets hosted backend resources such as WordPress EC2 instances and Amazon RDS database instances. An Internet Gateway (IGW) was attached to the VPC to provide internet connectivity for resources deployed within public subnets. A NAT Gateway was configured to allow outbound internet access for resources within private subnets without exposing them directly to the internet.

Separate route tables were configured for public and private subnets. Public subnet traffic was routed through the Internet Gateway, while private subnet traffic used the NAT Gateway for outbound connectivity. This architecture improved security by isolating backend infrastructure components from direct public access.

B. RDS Database Configuration

The database layer was implemented using Amazon Relational Database Service (RDS) with the MySQL engine [5]. Amazon RDS was selected to provide a managed database solution with automated backups, simplified maintenance, improved availability, and enhanced scalability compared to self-managed databases hosted on EC2 instances.

The primary RDS instance was deployed within private subnets to prevent direct public access. Security groups were configured using least privilege principles so that only the WordPress EC2 instances could communicate with the database layer. Multi-AZ deployment was enabled to improve fault tolerance by maintaining a synchronous standby replica in a separate Availability Zone.

In addition to the Multi-AZ configuration, a read replica was configured to improve read scalability and database redundancy. The WordPress application was connected securely to the primary RDS instance using the database endpoint and credentials configured during deployment.

C. WordPress Application Deployment

The WordPress application was deployed using Amazon EC2 instances running Amazon Linux 2 [6], [7]. Apache HTTP Server, PHP, MySQL client utilities, and required WordPress dependencies were installed automatically using EC2 user data scripts during instance launch.

After the EC2 instance was configured successfully, the WordPress installation process was completed through the web-based setup interface and connected to the Amazon RDS MySQL database. Once the deployment was fully configured, an Amazon Machine Image (AMI) was created from the EC2 instance and later used within the Auto Scaling Group launch template to ensure deployment consistency across newly launched instances.

The deployed WordPress application was accessible through the Application Load Balancer as shown in Figure 2.

D. Application Load Balancer and Auto Scaling

To improve scalability and availability, an Application Load Balancer and Auto Scaling Group were configured for the WordPress infrastructure [8]–[10]. The Application Load Balancer distributed incoming HTTP traffic across multiple EC2 instances while continuously performing health checks to ensure traffic was only routed to healthy instances.

The Auto Scaling Group used a launch template based on the previously configured Amazon Machine Image (AMI). Scaling policies were configured using CPU utilisation thresholds to dynamically launch or terminate EC2 instances depending on workload demand. The infrastructure was configured with a minimum capacity of one instance, desired capacity of one instance, and maximum capacity of three instances.

Scale-out policies launched additional EC2 instances when CPU utilisation exceeded 70%, while scale-in policies removed unnecessary instances when CPU utilisation dropped below 25%. This improved both resource efficiency and application availability during varying workload conditions.

E. S3 Media Offloading

Amazon Simple Storage Service (S3) was integrated into the WordPress infrastructure to provide scalable object storage for uploaded media files [11]. Instead of storing media directly on EC2 instances, the WordPress Offload Media plugin was configured to store uploaded content within an Amazon S3 bucket.

This approach reduced dependency on local EC2 storage and improved compatibility with Auto Scaling because newly launched instances could access the same centrally stored media files. The S3 bucket was configured with restricted public access and secure IAM role permissions to improve security and simplify authentication within the AWS Learner Lab environment.

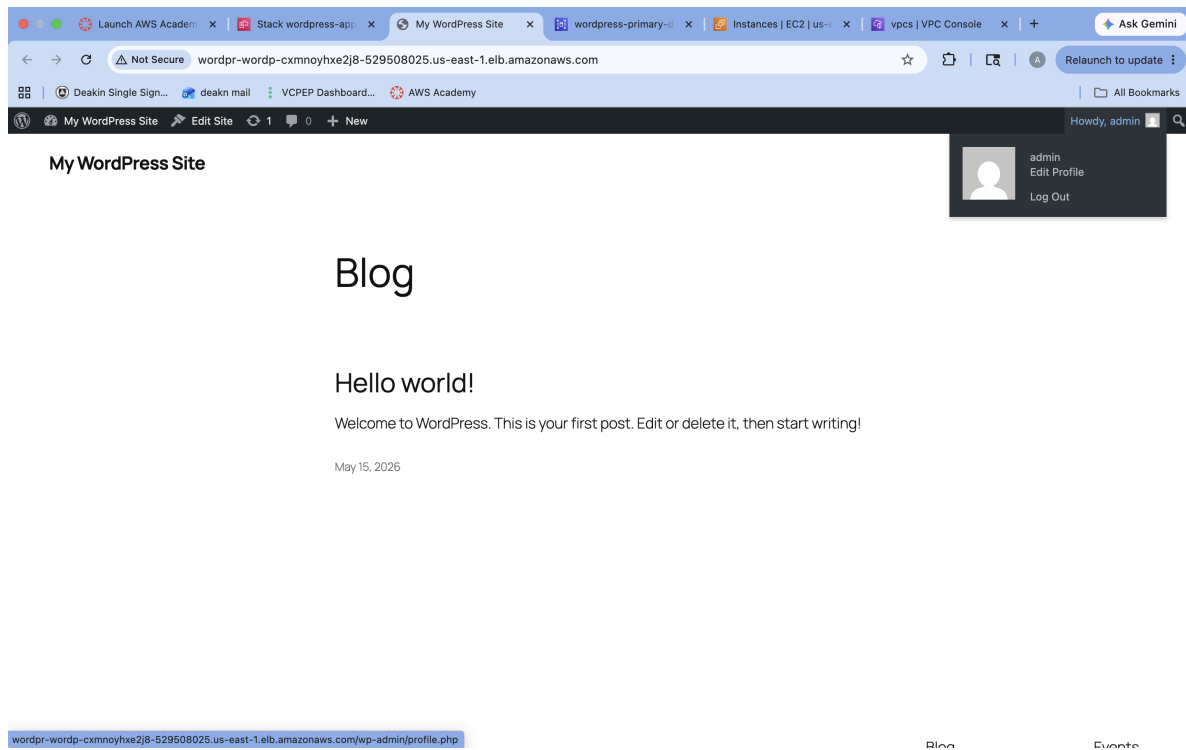


Fig. 2. Successfully Deployed WordPress Application

F. High Availability Testing

High availability testing was performed to validate the fault tolerance capabilities of the deployed infrastructure. One of the running WordPress EC2 instances was manually terminated through the AWS Management Console to simulate an infrastructure failure scenario.

After the instance was terminated, the Auto Scaling Group automatically detected that the desired capacity had fallen below the configured threshold and launched a replacement EC2 instance using the predefined launch template. The new instance was then automatically registered with the Application Load Balancer target group.

This demonstrated that the infrastructure could automatically recover from instance failures without requiring manual intervention, improving overall reliability and reducing service disruption.

Figure 3 shows the automatic EC2 replacement process during high availability testing.

G. CloudFormation Infrastructure Automation

Infrastructure automation was implemented using AWS CloudFormation and Infrastructure as Code (IaC) principles [3], [12]. CloudFormation templates written in YAML were used to automate the deployment and configuration of networking and application resources.

The infrastructure was separated into two independent CloudFormation stacks. The first stack focused on networking resources including the VPC, subnets, route tables, Internet Gateway, and NAT Gateway. The second stack focused on application resources including EC2 launch templates, Auto Scaling Groups, Application Load Balancer configuration, Amazon RDS deployment, and security groups.

Separating the infrastructure into modular CloudFormation stacks improved deployment management, reusability, and infrastructure maintainability while reducing manual configuration effort and deployment inconsistencies.

IV. DISCUSSION AND REFLECTIONS

The project successfully demonstrated the deployment of a highly available and scalable WordPress application using Amazon Web Services. By integrating services such as Amazon EC2, Amazon RDS, Amazon S3, Application Load Balancer, Auto Scaling Groups, Virtual Private Cloud, and AWS CloudFormation, the final infrastructure achieved the primary objectives of scalability, fault tolerance, infrastructure automation, and secure resource separation. The completed system provided a significantly more resilient architecture compared to a traditional single-server deployment model.

One of the major benefits of the implemented architecture was the separation of infrastructure into multiple layers including networking, application, and database components. The use of private subnets for backend services improved security by

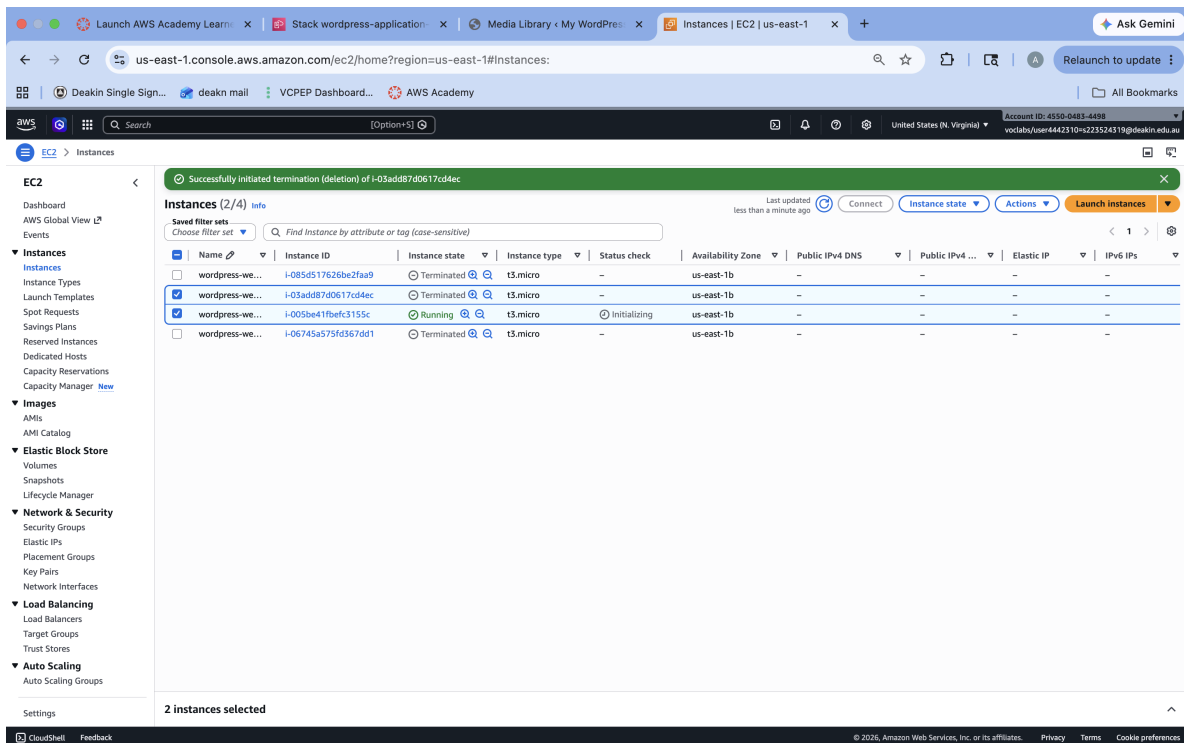


Fig. 3. Automatic EC2 Instance Replacement Using Auto Scaling

preventing direct internet exposure to EC2 application instances and database resources. In addition, the use of an Application Load Balancer and Auto Scaling Group improved availability and scalability by distributing traffic and automatically replacing failed EC2 instances when required.

During implementation, several practical challenges were encountered. Initially, the project specification recommended the use of `t2.micro` EC2 instances. However, repeated stability and performance issues were experienced during WordPress installation and service configuration, including unexpected crashes and resource exhaustion under workload conditions. To improve stability and reliability, the deployment was migrated to `t3.micro` instances, which provided improved performance and more consistent operation throughout the project implementation and testing phases.

Another significant challenge involved configuring WordPress media uploads to integrate correctly with Amazon S3. Initially, the implementation attempted to use AWS access keys directly within the WordPress Offload Media plugin configuration. However, due to security restrictions and credential management limitations within the AWS Learner Lab environment, this approach was unreliable and difficult to maintain. As a result, the implementation was modified to use IAM roles attached directly to the EC2 instances, allowing the WordPress application to securely access the S3 bucket without storing static access credentials within the application configuration. Although the setup process required substantial troubleshooting and experimentation, the final solution improved security and aligned more closely with AWS best practices.

The project also highlighted the practical advantages of Infrastructure as Code (IaC) using AWS CloudFormation. Automating infrastructure deployment through YAML templates significantly reduced manual configuration effort and improved deployment consistency. Separating the infrastructure into independent networking and application stacks also improved modularity and lifecycle management. The implemented CloudFormation templates were capable of recreating the majority of the deployed infrastructure automatically, demonstrating the benefits of infrastructure automation within cloud computing environments.

Additional automation ideas were also explored during the project. One planned enhancement involved storing the CloudFormation YAML templates within AWS CodeCommit and creating a deployment pipeline that would automatically create or update CloudFormation stacks whenever changes were pushed to the repository. This would have introduced a more complete CI/CD-style infrastructure deployment workflow. However, due to service limitations and restricted permissions within the AWS Learner Lab environment, the implementation of this automation pipeline could not be completed successfully.

Overall, the project provided valuable practical experience in designing and deploying cloud infrastructure using AWS services. The implemented architecture aligns with several AWS Well-Architected Framework principles including operational excellence, reliability, security, and performance efficiency. In addition, the project highlighted the importance of infrastructure planning, security configuration, service integration, and fault tolerance within real-world cloud computing environments.

V. CONCLUSION

This project successfully demonstrated the design and deployment of a highly available WordPress hosting environment using Amazon Web Services. The implemented solution integrated multiple AWS services including Amazon EC2, Amazon RDS, Amazon S3, Application Load Balancer, Auto Scaling Groups, Virtual Private Cloud, and AWS CloudFormation to create a scalable, secure, and fault-tolerant cloud infrastructure.

The final architecture improved application availability and reliability by distributing resources across multiple Availability Zones, implementing automated load balancing and scaling, and deploying the database layer with Multi-AZ and read replica configurations. Media storage integration with Amazon S3 further improved scalability and reduced dependency on local EC2 instance storage. In addition, Infrastructure as Code principles were successfully applied through AWS CloudFormation to automate infrastructure provisioning and improve deployment consistency.

High availability testing confirmed that the infrastructure could automatically recover from EC2 instance failures through Auto Scaling Group replacement mechanisms. The project also provided practical experience in networking configuration, infrastructure automation, cloud security, and managed service integration within AWS cloud environments.

Overall, the project demonstrated how modern cloud computing services can be combined to deliver resilient and scalable web application infrastructure while reducing manual management complexity and improving operational efficiency.

VI. PRESENTATION

The video presentation for this project is available at: <https://youtu.be/l8ZRqyIIsMU>

APPENDIX A INFRASTRUCTURE CONFIGURATION PARAMETERS

This appendix summarises the key infrastructure configuration parameters used throughout the AWS deployment. The tables below provide important networking, compute, database, scaling, and storage configurations implemented during the project.

A. VPC and Networking Configuration

TABLE I
VPC NETWORKING CONFIGURATION PARAMETERS

Parameter	Value
AWS Region	us-east-1
VPC CIDR Block	192.168.0.0/16
Public Subnet 1	192.168.0.0/24
Public Subnet 2	192.168.1.0/24
Private Subnet 1	192.168.2.0/24
Private Subnet 2	192.168.3.0/24
Availability Zones	us-east-1a, us-east-1b
Internet Gateway	Enabled
NAT Gateway	Enabled
Public Route Table	IGW Routing
Private Route Table	NAT Gateway Routing

B. EC2 and Auto Scaling Configuration

TABLE II
EC2 AND AUTO SCALING CONFIGURATION PARAMETERS

Parameter	Value
EC2 Instance Type	t3.micro
Operating System	Amazon Linux 2
Web Server	Apache HTTP Server
Application Platform	WordPress
Launch Template	Configured
Auto Scaling Group	Enabled
Minimum Capacity	1
Desired Capacity	1
Maximum Capacity	3
Scale-Out Policy	CPU Utilisation > 70%
Scale-In Policy	CPU Utilisation < 25%
Health Checks	Enabled
AMI Creation	Enabled

C. Application Load Balancer Configuration

TABLE III
APPLICATION LOAD BALANCER CONFIGURATION PARAMETERS

Parameter	Value
Load Balancer Type	Application Load Balancer
Scheme	Internet Facing
Target Type	EC2 Instances
Health Check Protocol	HTTP
Listener Port	80
Target Group	Configured
Cross-Zone Load Balancing	Enabled
Subnet Placement	Public Subnets

D. RDS Database Configuration

TABLE IV
AMAZON RDS CONFIGURATION PARAMETERS

Parameter	Value
Database Engine	MySQL
Deployment Type	Multi-AZ
Read Replica	Enabled
Public Accessibility	Disabled
Subnet Placement	Private Subnets
Automated Backups	Enabled
Database Security Groups	Configured
Storage Type	General Purpose SSD

E. Amazon S3 Configuration

TABLE V
AMAZON S3 MEDIA STORAGE CONFIGURATION

Parameter	Value
Storage Service	Amazon S3
Storage Class	Standard
Public Access	Blocked
Media Integration	WordPress Offload Media Plugin
Authentication Method	IAM Role
Bucket Versioning	Disabled
Purpose	WordPress Media Storage

F. CloudFormation Infrastructure Automation

TABLE VI
CLOUDFORMATION INFRASTRUCTURE AUTOMATION PARAMETERS

Parameter	Value
Infrastructure as Code	Enabled
Template Format	YAML
Networking Stack	Separate Stack
Application Stack	Separate Stack
Stack Dependency	Export/Import Outputs
Reusable Deployment	Supported
Automated Resource Provisioning	Enabled

APPENDIX B AWS CONSOLE SCREENSHOTS

This appendix contains additional AWS Management Console screenshots captured during the implementation and testing of the highly available WordPress infrastructure.

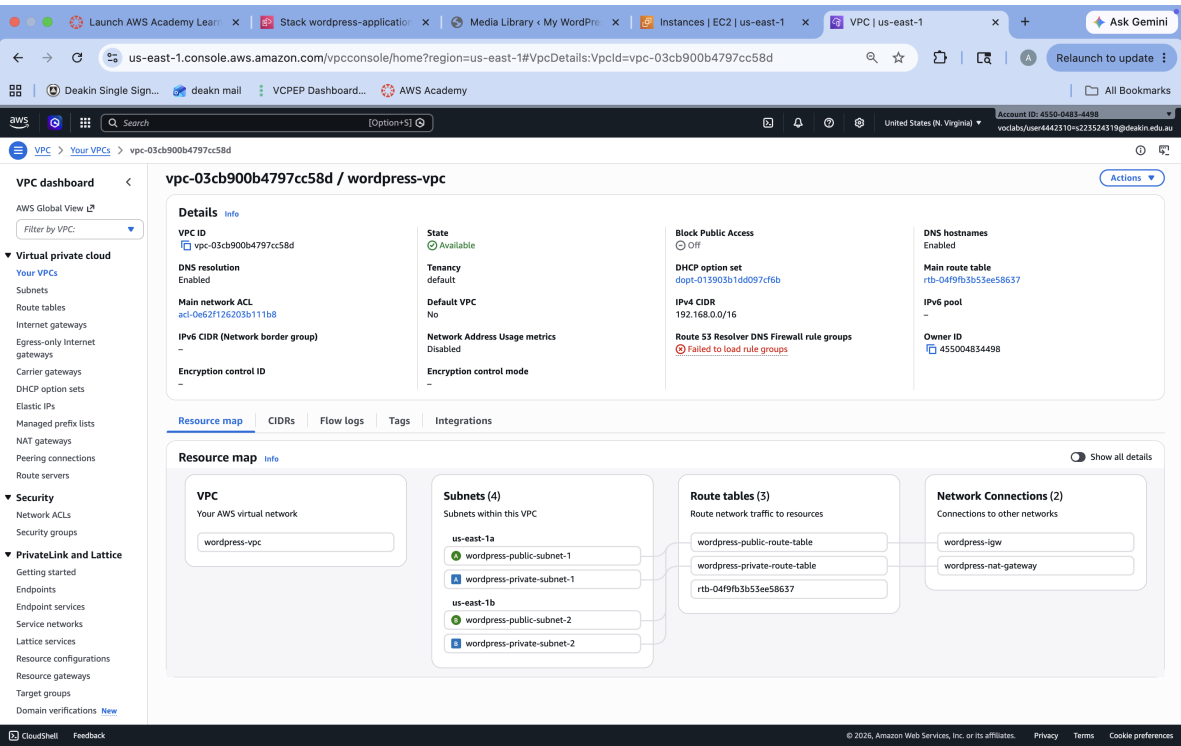


Fig. 4. VPC Networking Architecture Configuration

Figure 4 illustrates the networking architecture implemented within the AWS environment, including the custom VPC, public and private subnets, routing configuration, NAT Gateway, and the placement of major infrastructure components across multiple Availability Zones.

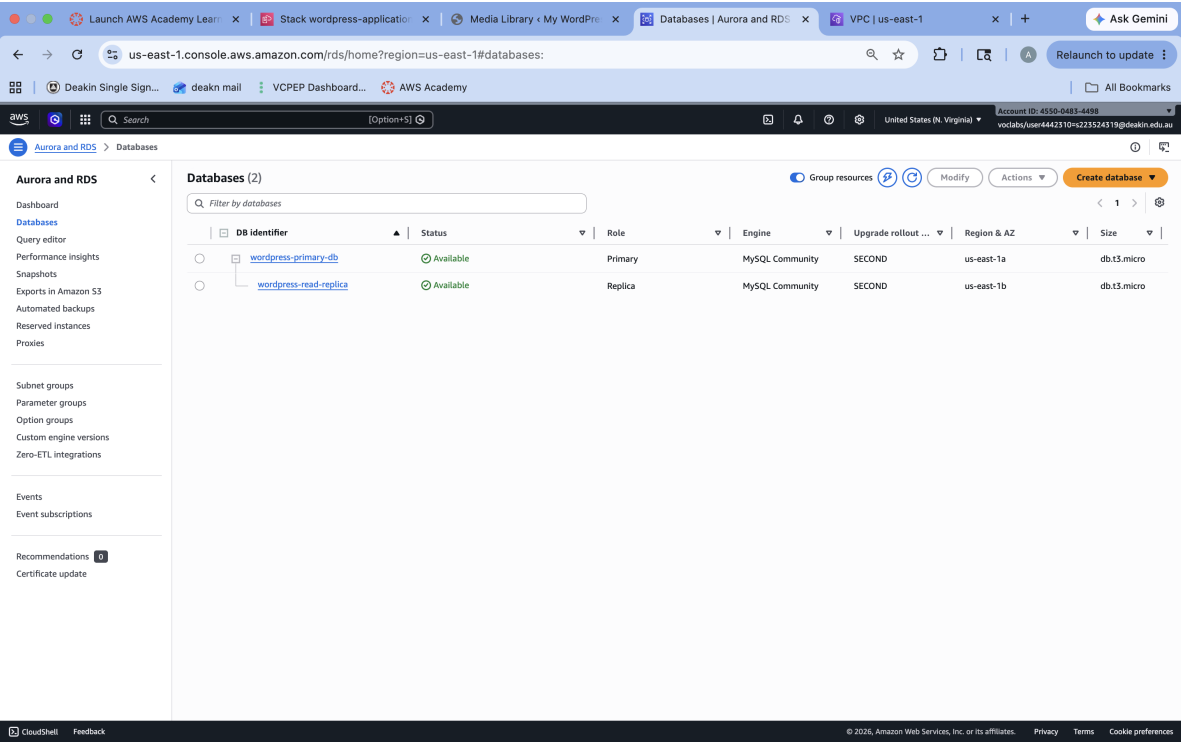


Fig. 5. Amazon RDS Primary Instance and Read Replica Configuration

Figure 5 shows the Amazon RDS MySQL deployment configured with a primary database instance and read replica to improve fault tolerance, redundancy, and database availability.

The screenshot displays the AWS Management Console for an Application Load Balancer. The left sidebar shows the navigation menu with categories like EC2, Instances, Images, Elastic Block Store, Network & Security, Load Balancing, and Auto Scaling. The main content area is titled 'wordpr-WordP-cXMNOyhXe2j8' and includes a 'Details' section with information such as Load balancer type (Application), Scheme (Internet-facing), Status (Active), Hosted zone (Z355XDOTRQ7X7K), VPC (vpc-03c900b4797cc58d), Availability Zones (us-east-1b), Load balancer IP address type (IPv4), Date created (May 15, 2026), Load balancer ARN, and DNS name. Below this is a 'Resource map' section showing the architecture: Listeners (1) with HTTP:80, Rules (1) with Priority default, Target groups (1) with Instance, HTTP, and Targets (1) with Port 80. The status of the target is shown as 'Healthy'.

Fig. 6. Application Load Balancer Configuration

Figure 6 presents the Application Load Balancer configuration used to distribute incoming traffic across the WordPress EC2 instances while performing health checks on registered targets.

The screenshot displays the AWS Management Console for an Auto Scaling Group. The left sidebar shows the navigation menu. The main content area is titled 'wordpress-application-stack-WordPressAutoScalingGroup-JkRWOYhA71xj' and includes a 'Capacity overview' section showing Desired capacity (1), Scaling limits (1 - 3), Desired capacity type (Units (number of instances)), and Status (At desired capacity). Below this is a 'Details' section with tabs for Launch template, Network, and Automatic scaling. The Launch template section shows the Launch template (wordpress-application-stack-WordPressAutoScalingGroup-JkRWOYhA71xj), AMI ID (ami-03fd597129d2144d), Instance type (t3.micro), Security group IDs (sg-079254d5885cb5341), Key pair name (vockey), and Owner (arn:aws:sts::455004834498:assumed-role/LabRole/AWSCloudFormation). The Network section shows Availability Zones (us-east-1a, us-east-1b), Subnet ID (subnet-0179a1da2c9af8189), and Availability Zone distribution (Balanced best effort).

Fig. 7. Auto Scaling Group Configuration

Figure 7 illustrates the Auto Scaling Group configuration including the minimum, desired, and maximum instance capacities along with the scaling policies configured for dynamic workload management.

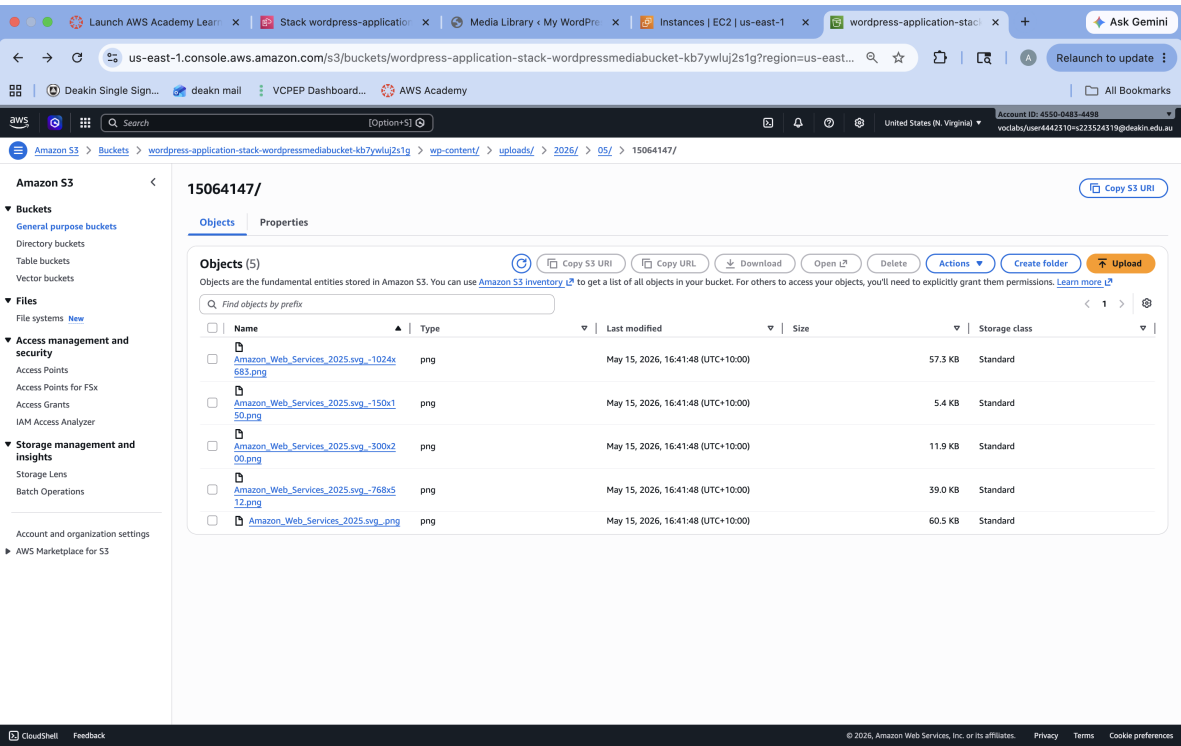


Fig. 8. Amazon S3 Offloaded Media Files

Figure 8 shows the Amazon S3 bucket used to store WordPress media uploads through the Offload Media plugin integration.

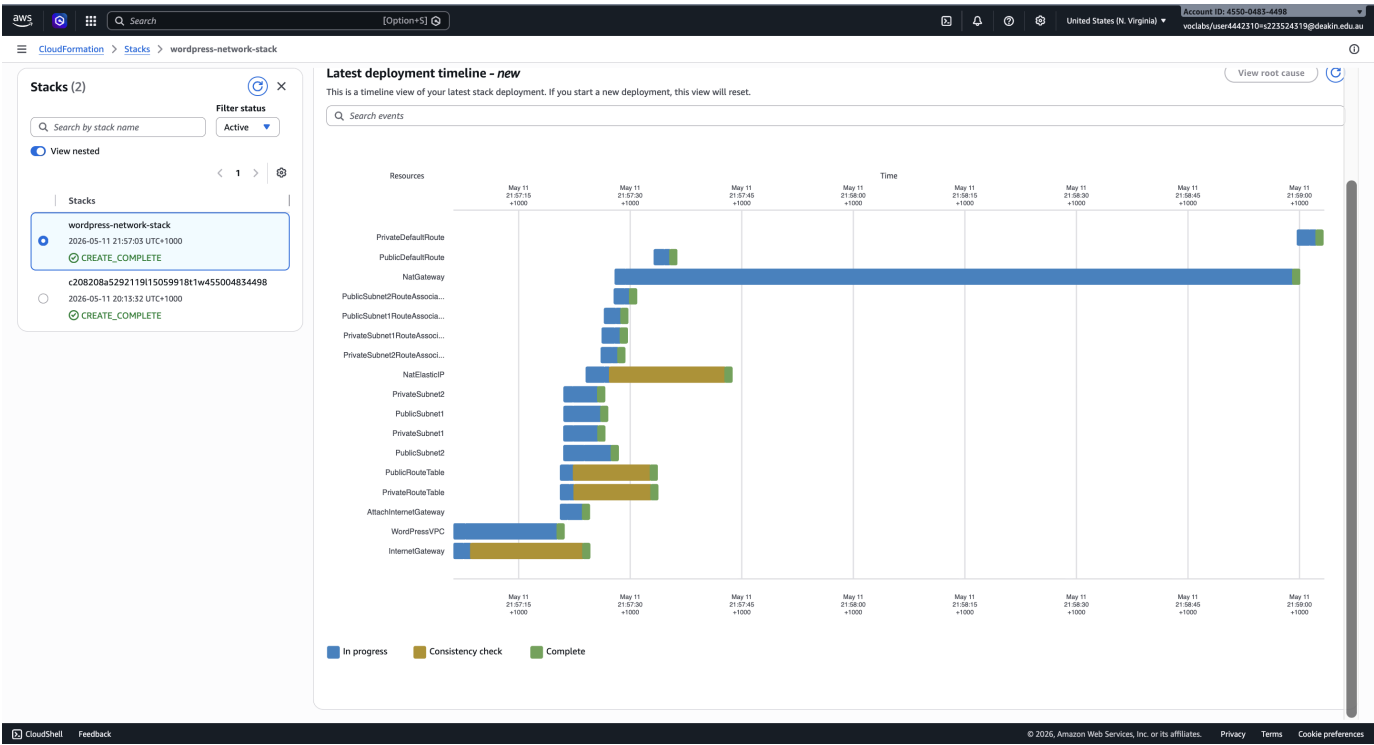


Fig. 9. CloudFormation Networking Stack Deployment

Figure 9 presents the successful deployment timeline for the CloudFormation networking stack responsible for provisioning VPC and networking resources.

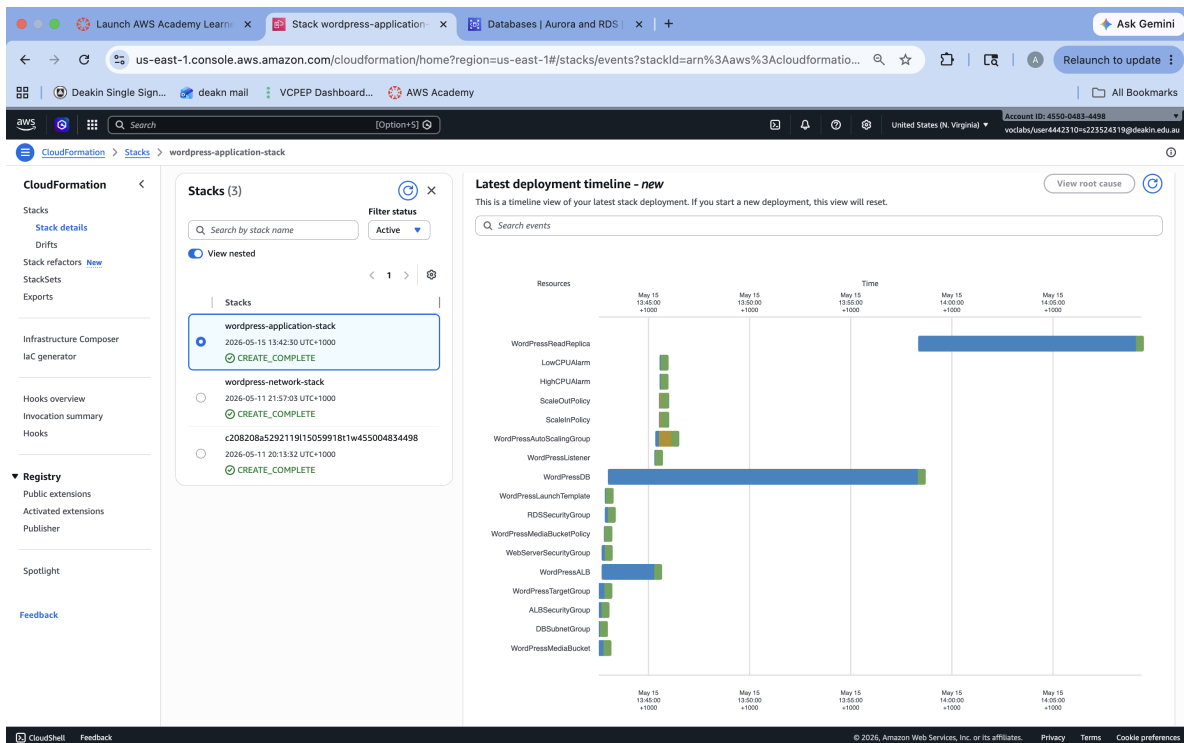


Fig. 10. CloudFormation Application Stack Deployment

Figure 10 illustrates the successful deployment of the CloudFormation application stack responsible for provisioning compute, scaling, database, and application infrastructure resources.

REFERENCES

- [1] Amazon Web Services, "Aws well-architected framework," 2026. [Online]. Available: <https://docs.aws.amazon.com/wellarchitected/latest/framework/welcome.html>
- [2] WordPress.org, "Wordpress documentation," 2026. [Online]. Available: <https://wordpress.org/documentation/>
- [3] Amazon Web Services, "Aws cloudformation user guide," 2026. [Online]. Available: <https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/Welcome.html>
- [4] —, "Aws::ec2::vpc resource reference," 2026. [Online]. Available: <https://docs.aws.amazon.com/AWSCloudFormation/latest/TemplateReference/aws-resource-ec2-vpc.html>
- [5] —, "Amazon rds user guide," 2026. [Online]. Available: <https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/Welcome.html>
- [6] —, "Amazon ec2 user guide," 2026. [Online]. Available: <https://docs.aws.amazon.com/ec2/>
- [7] —, "Aws::ec2::instance resource reference," 2026. [Online]. Available: <https://docs.aws.amazon.com/AWSCloudFormation/latest/TemplateReference/aws-resource-ec2-instance.html>
- [8] —, "Elastic load balancing user guide," 2026. [Online]. Available: <https://docs.aws.amazon.com/elasticloadbalancing/>
- [9] —, "Amazon ec2 auto scaling user guide," 2026. [Online]. Available: <https://docs.aws.amazon.com/autoscaling/ec2/userguide/>
- [10] —, "Aws auto scaling cloudformation reference," 2026. [Online]. Available: https://docs.aws.amazon.com/AWSCloudFormation/latest/TemplateReference/AWS_AutoScaling.html
- [11] —, "Amazon s3 user guide," 2026. [Online]. Available: <https://docs.aws.amazon.com/AmazonS3/latest/userguide/Welcome.html>
- [12] —, "Aws cloudformation template reference," 2026. [Online]. Available: <https://docs.aws.amazon.com/AWSCloudFormation/latest/TemplateReference/>